AWS EC2 Secure Web Server: Project Report

Varshini Murali | Murdoch University | ICT171 Cloud Server Project

Summary

I built and hosted a recipe website, the World Recipe Collection, on my own AWS EC2 server. I set up the web server, a domain name and HTTPS myself, and wrote a Bash script that backs up the website, cleans up old backups and logs every run. The site is live at https://nicerecipes.duckdns.org. The project showed me that running a server means thinking about security and recovery from the start, not just getting the site online.

Problem

I needed to host a website on the cloud and make it secure, without using a managed hosting service or a pre-built image. Doing everything manually (Infrastructure as a Service) gave me full control of the server, but also meant I was responsible for its security, its domain name and what happens if files are lost or damaged.

Tools

Tool	What I used it for
AWS EC2 (t3.micro, Ubuntu 22.04 LTS)	The cloud server
SSH	Connecting to the server from my laptop
Apache	The web server that serves the website
HTML and CSS	The recipe website, one page per cuisine
DuckDNS	A free domain name pointing to the server
Certbot and Let's Encrypt	A free HTTPS certificate
Bash	The backup script
GitHub and Markdown	Documenting every step

Action

1. Launched an EC2 instance with Ubuntu 22.04 LTS. In the security group I allowed only ports 22 (SSH), 80 (HTTP) and 443 (HTTPS), and used a key pair for SSH access.
2. Connected to the server with SSH.
3. Installed Apache and set it to start automatically.
4. Built the World Recipe Collection website with HTML and CSS and copied it to Apache's web root, then set the correct file owner.

5. Registered a DuckDNS subdomain and created an A record pointing to the server's public IP address.
6. Used Certbot to get a Let's Encrypt certificate, install it and redirect all HTTP traffic to HTTPS.
7. Wrote `backup.sh`, which compresses the website into a timestamped archive, deletes backups older than 7 days, and logs the date, time and result of every run.
8. Documented each step in a public GitHub repository with commits made over several weeks.

Findings

- The website is live and loads at the DuckDNS domain.
- HTTPS works, the browser shows a padlock, and HTTP requests are redirected to HTTPS.
- The domain pointed to the server within a few minutes of saving the DNS record.
- Each run of the backup script creates a timestamped archive and writes a line to the log, so there is a record of when backups ran and whether they succeeded.
- I run the backup script by hand. It does not yet run on a schedule.

Risk and security notes

Area	Current state	Risk
SSH (port 22)	Open for administration	Medium. Port 22 should be limited to my own IP address.
HTTPS	Enabled with a valid certificate	Low
Backups	Created manually and stored on the same server	Medium. If the server fails, the backups could be lost too.
Updates	Not yet scheduled	Medium. The server needs regular updates.

Recommendations

1. Restrict SSH access in the security group to my own IP address.
2. Add a cron job so the backup script runs on a schedule.
3. Copy backups to separate storage, such as an S3 bucket.
4. Run regular updates and check that the Certbot certificate renews.

Lessons learned

- Building everything myself, from the virtual machine to Apache, gave me a much better understanding of how web hosting works.
- HTTPS encrypts all traffic between a visitor and the server, and Certbot makes it simple to set up properly.
- A backup is only useful if it is logged, tested and stored somewhere safe.
- Security is part of the build, from opening only the ports you need to protecting your key file.

Evidence

- Live site: https://nicerecipes.duckdns.org
- GitHub repository: vm553/ict171-cloud-server-project